

Thrust Vector Control for Mars Lander

Team dm-001

Member 1: PVNS Meenakshi

Member 2: Amadala Sathvik

Member 3: Paholpreat Kaur

1 Introduction

Landing a spacecraft safely on the surface of Mars is a complex challenge that requires precise control of both trajectory and attitude during descent. This project focuses on modeling and simulating the descent phase of a Mars lander using a central engine equipped with gimbaled thrust vector control (TVC). To explore the best propulsion layout, two configurations were initially considered: a single central engine and a system of distributed thrusters. After evaluating factors like mechanical complexity, weight, and integration effort, the central engine approach was chosen for its simplicity and lower system overhead.

The lander uses a bipropellant system with Monomethylhydrazine (MMH) as fuel and Nitrogen Tetroxide (NTO) as oxidizer. Tanks are symmetrically positioned around the central axis to help lower the center of gravity and maintain stability. Several key sensors support the descent phase: an Inertial Measurement Unit (IMU) for orientation, a radar altimeter for height estimation, a barometric sensor for atmospheric data, and touchdown sensors to detect landing contact.

Various TVC techniques were studied, including basic methods like jet vanes and secondary fluid injection, but the system ultimately uses the more advanced gimbaled thrust approach. In this setup, the engine nozzle is tilted via actuators to control pitch and yaw.

To replicate the lander's behavior in a simulated environment, the system is divided into five main subsystems in Simulink: Guidance, Control System, Actuators, Lander Dynamics, and Sensors. The control system uses a PID controller to minimize errors between actual and desired states by adjusting throttle and gimbal angles. The actuator block then computes the thrust direction and resulting torques, accounting for the changing center of mass due to fuel consumption and heat shield separation. The lander dynamics are modeled using a custom 6-degree-of-freedom (6DOF) block based on quaternions, which handles forces, torques, mass changes, and even aerodynamic drag, modeled with an exponential Mars atmosphere profile.

To enhance accuracy and robustness, a Kalman Filter will be incorporated in future stages to fuse sensor data and reduce noise, helping to ensure a smooth and controlled landing despite uncertainties in real-world conditions.

2 Propulsion Distribution

2.1 Central Engine

A large main engine is generally placed at the centre of mass which can be gimbaled for thrust vectoring.

Basic layout of central engine

The main engine should be aligned with the lander's vertical axis. The fuel and oxidiser tanks are to be placed around the central engine. A Gimbal mount, which is a mechanical structure that allows the main engine to rotate in one or more directions so that the thrust vector can be redirected, is placed directly at the base of the central engine where it connects to the vehicle which is between the nozzle and thrust structure. Then we have RCS (Reaction Controller System) which should be placed around the top extremities for precise roll control (gimbal only controls yaw and pitch; RCS is used for roll control).

Pros and cons of central engine

- It has the most simple mechanical layout.
- As a single main engine is used it is usually light in weight.
- It is easier to assemble and integrate.
- If the main engine fails, the lander will not reach the destination as there is no other main engine working.
- Usually the main engine is gimbaled and we need extra engines (RCS) for complete 3-axis control.
- This kind of propulsion system leads to less complexity because of less hardware and software overhead.

Basic Working Mechanism

In the pre-descent phase that happens near the Mars orbit the onboard activities are: IMU starts to record the descent acceleration and orientation, RADAR altimeter measures altitude and the engine initializes the descent sequence. When the lander reaches an altitude approximately of 2km to 500m, the central engine starts to ignite. The engine is supplied with propellant (MMH+NTO). Throttle control refers to controlling thrust output based on deceleration. If the lander tilts the gimbal tilts the engine nozzle, this shifts the thrust vector to steer the lander back on track. IMU detects pitch and yaw errors, PID controller minimizes the error. Gimbal actuators adjust the nozzle angle typically (5–15 degrees) and the thrust vector is redirected.

2.2 Distributed Thrusters

Multiple smaller engines or thrusters are distributed symmetrically around the base, along the sides of the lander. Each of the engines can be gimbaled to control the attitude.

Basic layout of the distributed thrusters

There are 3 to 8 small thrusters placed radially on the base of the lander which will be symmetric about the Centre of Mass. Fuel tanks and oxidizers are placed at the centre of mass. RCS are again placed around the lander. Mounting frames are fixed to engines.

Pros and cons of distributed thrusters

- Even if one engine fails the other engines will still function and will not lead to mission failure.
- It is a heavier structure due to multiple engines.
- It has to be dealt with very complex control logic and complex guidance.
- Placement of many engines to the edge leads to increased complexity in designing legs.
- Plumbing of the propellant pipes will be difficult because of the supply to many engines.

Basic Working Mechanism

During powered descent, a Mars lander with a distributed thruster propulsion system uses several small thrusters placed symmetrically around its body to reduce vertical velocity and maintain stability. These thrusters, usually located near the center of mass, fire together to control descent speed, while smaller reaction control system (RCS) thrusters manage orientation by adjusting pitch, roll, and yaw through asymmetric firings. Some thrusters may be angled or pulsed to enable lateral movement, allowing the lander to avoid hazards and make fine position corrections during the final phase. As it approaches the surface, the system transitions to low-thrust, precise firings for a smooth touchdown. All thruster activity is continuously adjusted in real-time using feedback from onboard sensors such as IMUs, altimeters, and LIDAR, with control algorithms ensuring coordinated thrust for a stable and accurate landing.

Analyzing the pros and cons of each system, we proceed to use a central engine system to avoid complex plumbing system, complicated control logic, and intricate guidance.

3 Structural Design

We are using a truncated shape for our Mars lander. This shape offers more stability due to a wider base and lower positioning of the centre of gravity. The truncated geometry provides better base contact and is more adaptable to rough terrains. It is also easier to manufacture and provides a better configuration for engine mounting and Thrust Vector Control (TVC) integration.

The gimbal engine rotates in two directions: pitch and yaw. In this design, the entire engine—including the combustion chamber—rotates. The typical gimbal angle range is about 5° to 10° . This approach is inspired by existing missions such as *Viking* and the *Apollo Lunar Module*, where the entire engine is gimbaled.

To accommodate the motion of the engine, flexible propellant lines are used. Selecting appropriate actuators is critical for the controlled rotation of the gimbaled engine. We have chosen electromechanical actuators over hydraulic actuators for several reasons:

- Electromechanical actuators are better suited for operating in extreme temperature ranges from -20°C to $+20^\circ\text{C}$, as encountered on Mars.
- They perform reliably in low-pressure environments.
- They feature a simple construction consisting of an electric motor and gearbox.
- The control system for electromechanical actuators is more precise and accurate compared to hydraulic systems.

These advantages make electromechanical actuators more suitable for space applications, especially for autonomous thrust vector control in planetary descent missions.

4 Placement Of Tank

The lander uses a bipropellant system with Monomethylhydrazine (MMH) as fuel and Nitrogen Tetroxide (NTO) as oxidizer. Two cylindrical tanks with hemispherical ends are mounted symmetrically on either side of the central engine. This geometry ensures pressure resistance and efficient volume use. To meet the lander height constraint of 2.5 meters, the tanks are placed laterally near the engine. This positioning lowers the center of gravity, enhancing stability during descent and landing. Symmetrical alignment close to the thrust axis minimizes torque and pitch disturbances. Tanks are shielded from direct sunlight and engine exhaust to maintain propellant integrity. Thermal and structural protection is provided where necessary.

5 Key Sensors and System Placement

IMU (Inertial Measurement Unit) is placed inside the lander (or rover) aligned with the central axis. IMU should be provided with shielding from thermal radiation and protected from vibration. Placement of IMU is critical for guidance, navigation, and control. Placing the IMU near CG minimizes error in acceleration and angular measurements. It ensures measurements reflect true body motion and not local vibrations.

Radar altimeter measures the distance to the Martian surface by sending radio waves downward and timing their return. The radar altimeter must be mounted on the bottom of the lander. No tank or legs should block its field of view. It should be placed at the outer edge of the lander baseplate between two landing legs.

Barometric pressure sensor is important for understanding Mars' thin atmosphere during descent and on the surface. It performs critical operations like parachute ejection and studying the weather and climate on Mars. The sensor must be directly exposed to weather, so it can't be placed inside the structure. It is placed on the side or top of the lander body. The sensor should be placed away from the downward airflow or exhaust and dust from the atmosphere.

Heat shield and touchdown sensors are used for critical mission events like entry heat and landing confirmations on Mars. Heat shield sensors are placed at the outer surface of the heat shield, and touchdown sensors are placed in each landing leg to end the descent phase.

6 Thrust Vector Control Method

6.1 Primitive Methods

Jet Vanes Thrust Vector Control

There are many different ways in which the direction of the resultant thrust vector can be manipulated. Jet vanes act as aerodynamic control surfaces, causing the jet stream centerline to deflect from the longitudinal axis of the missile; thrust vectoring is achieved in this manner. Jet vanes are placed in a high-velocity stream, which disrupts the flow. This turbulence introduces inefficiency in overall performance.

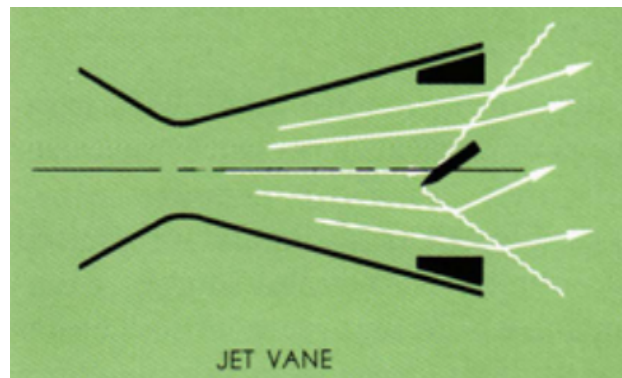


Figure 1: Jetvane

Secondary Injection Thrust Vector Control

Secondary Injection Thrust Vector Control technique is used to steer rockets or missiles by injecting a fluid, typically a gas or liquid, into the high-speed exhaust stream of the engine. This fluid is introduced sideways into the diverging section of the rocket nozzle through small injector ports. The imbalance introduced generates torque that controls yaw and pitch, helping to attain the desired trajectory while descending.

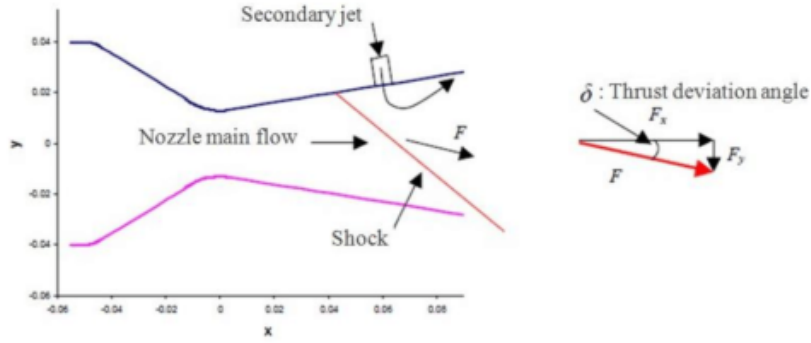


Figure 2: Secondary Injection TVC

6.2 Intermediate Advanced Methods

Gimbaled Thrust Vector Control

Most modern rockets and landers use a method called gimbaled thrust to steer during flight. In this system, either the entire engine or just the exhaust nozzle is mounted so it can swivel on two axes, pitch and yaw. By tilting the nozzle, the direction of the thrust is changed relative to the rocket's center of gravity. When the thrust is no longer aligned perfectly through the center of gravity, it creates a torque, causing the rocket to rotate or turn. For example, if the nozzle is angled slightly to the right, the nose of the rocket will begin to turn to the right. This pivoting is done using hydraulic or electromagnetic actuators, often referred to as Thrust Vector Control (TVC) actuators. These actuators are connected near the top of the combustion chamber, allowing precise control over the nozzle angle. By continuously adjusting this angle, the rocket can be steered smoothly during ascent or descent.

7 Simulation on Simulink Until Mid-Evaluation

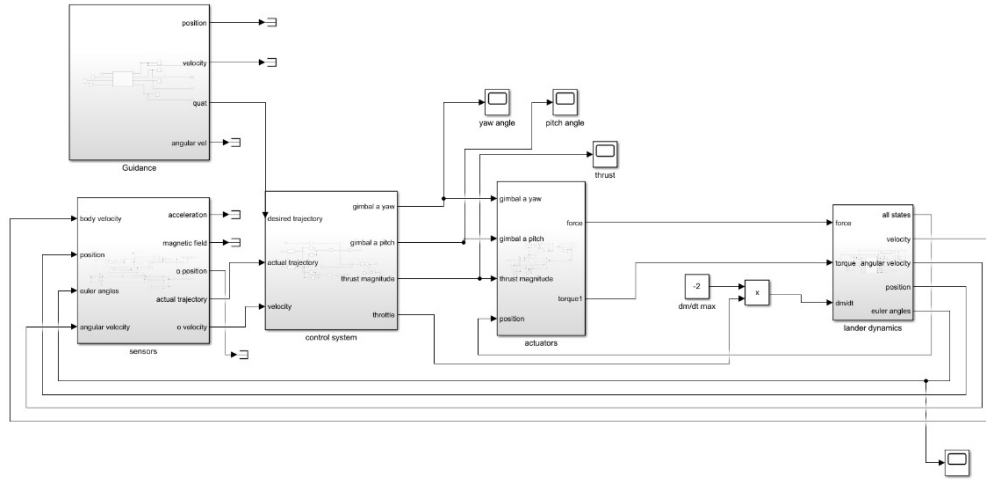


Figure 3: complete model

This project focuses on simulating and controlling the descent of a Mars lander that uses a gimballed engine and throttle control, guided by a Thrust Vector Control (TVC) system. The main goal is to ensure the lander touches down softly and safely on the Martian surface. To achieve this, the system continuously adjusts the direction and magnitude of the engine thrust based on real-time data about the lander's position, speed, and orientation.

To make the simulation manageable and realistic, we've broken the overall system into several key subsystems. Each of these represents a crucial part of how the lander behaves, senses its environment, and reacts during its descent. Together, these subsystems form a complete picture of the lander's dynamics and control strategy.

- GUIDANCE
- LANDER DYNAMICS
- CONTROL SYSTEM
- ACTUATORS
- SENSORS

All subsystems will work together in a feedback loop to guide the lander safely. To improve accuracy, we will add a Kalman Filter, which will combine sensor data and reduce noise for better state estimation and smoother control during descent.

The Sensors and Kalman Filter are yet to be added.

8 Implementation Inside the Subsystems

8.1 Control System

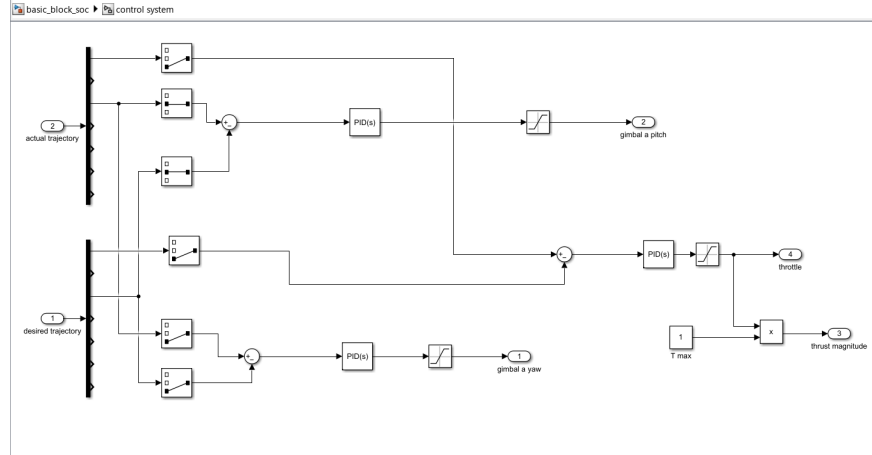


Figure 4: Controller subsystem

Objective: To compute the necessary gimbal angles for pitch and yaw to steer the thrust vector. Adjust the throttle to control the speed of vertical descent. Output commands that go to the actuators; these will affect the lander's dynamics.

Components and Functions:

- **Inputs:**
 - Desired trajectory (position, velocity, Euler angles, angular velocity in body frame (FROM THE GUIDANCE SUBSYSTEM))
 - Actual trajectory (feedback FROM THE LANDER DYNAMICS SUBSYSTEM)
- **Gimbal pitch and yaw loops:**
 - PID loops use the error between desired and actual positions to determine gimbal angles required to steer the thrust vector.
- **Throttle control:**
 - A third PID loop is used to regulate the throttle. The input is altitude error ($z_{\text{desired}} - z_{\text{actual}}$). The output of this PID gives a throttle fraction (0–1).
Scalar throttle command, used to scale the total thrust vector is obtained by the PID. You multiply this with a constant T_{max} (max thrust) to get the thrust magnitude.
- **Outputs:**

- Gimbal pitch
 - Gimbal yaw
 - Throttle which further gives us Thrust
- **Future extensions of controller subsystem:** To augment the control system, we suggest an extension in the future when throttle output is dynamically commanded to control attitude and descent rate simultaneously. A Throttle Control Subsystem will be specifically designed and incorporated into the current control architecture. The subsystem will:
 - Compensate net thrust vector to adjust attitude errors through gimbal actuation.
 - Modulate total thrust magnitude through the throttle command to decrease vertical descent speed smoothly, particularly during terminal descent.
 - This method allows for coupled control of rotational and translational dynamics, enhancing the stability and accuracy of the lander during descent.

8.2 Actuators

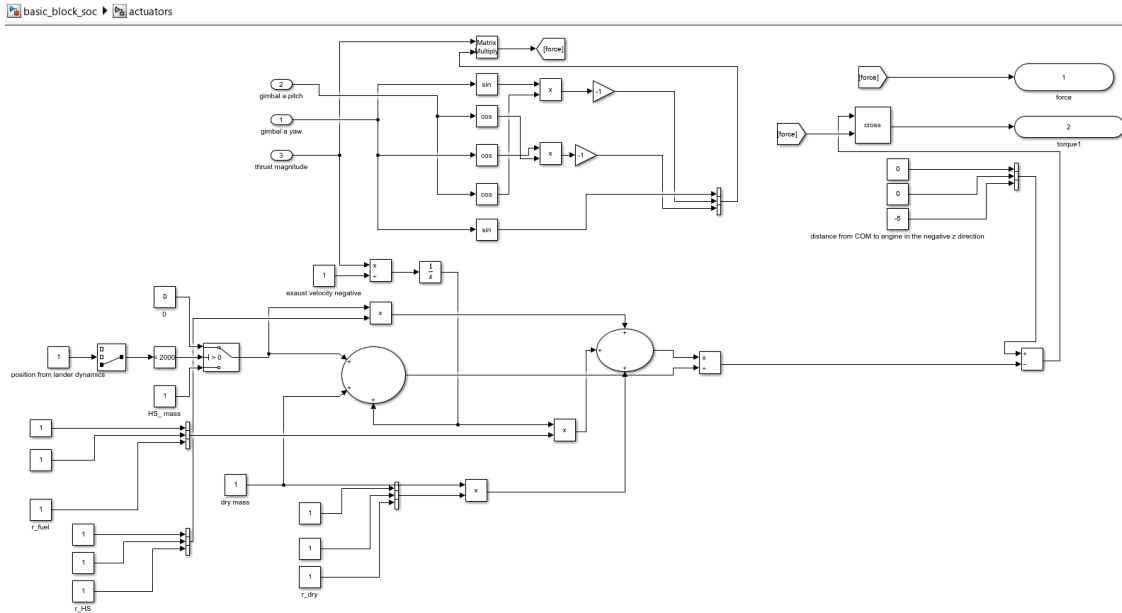


Figure 5: Actuators subsystem

Objective: To compute force and torque using certain rotation matrices that involve the gimbal pitch and gimbal yaw which are received by the controller subsystem.

Components and Functions:

- **Thrust:** Received from the controller subsystem and has to be converted into force.

- **Gimbal Pitch and Yaw:** Received from the controller subsystem and are used to build a composite rotation matrix which helps in rotating the thrust vector away from the vertical axis. (-ve z direction) A rotation matrix R is constructed using Euler angles as:

$$R = R_z \cdot R_y \cdot R_x$$

$$\mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi_c & \sin \phi_c \\ 0 & -\sin \phi_c & \cos \phi_c \end{bmatrix} \quad \mathbf{R}_z = \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R}_y = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}$$

In this model, only pitch and yaw are controlled, so the roll angle is to be set to zero, and the simplified rotation becomes:

$$R = R_z \cdot R_y$$

- **Force Calculation:** Gimbal angles are used to form a 3x1 direction matrix using R_x, R_y, R_z rotation matrices.

We assume the initial thrust to be in the negative z-axis direction. After the calculation, we get the rotated thrust direction vector.

$$\begin{bmatrix} \sin(\theta) \\ -\sin(\theta) \cos(\phi) \\ -\cos(\theta) \cos(\phi) \end{bmatrix}$$

We multiply the thrust from the control system with this matrix to get the force vector.

- **Torque Calculation:** Torque is computed via cross product:

$$\text{Torque} = \vec{r} \times \vec{F}$$

where $\vec{r}$ is the vector from COM to engine nozzle. but we should be mindful of the fact that COM keeps changing because of the mass variation due to the fuel consumption and separation of heat shield at a particular altitude, so, COM's position vector should be updated accordingly. This can take place as follows, using the basic Centre of mass formula:

$$\vec{r}_{\text{COM}} = \frac{m_{\text{dry}} \cdot \vec{r}_{\text{dry}} + m_{\text{fuel}} \cdot \vec{r}_{\text{fuel}} + m_{\text{hs}} \cdot \vec{r}_{\text{hs}}}{m_{\text{dry}} + m_{\text{fuel}} + m_{\text{hs}}}$$

Where:

- m_{dry} : dry mass
- m_{fuel} : fuel mass (updated dynamically by integrating $\frac{dm}{dt}$, obtained using throttle)

- m_{hs} : mass of heat shield
- $\vec{r}_{dry}$: position vector of the COM of dry mass
- $\vec{r}_{fuel}$: position vector of the COM of fuel
- $\vec{r}_{hs}$: position vector of the COM of heat shield

While working with this on simulink, we use a switch mechanism to control the variation of the lander's mass due to the heat shield's separation. This works such that the mass of the heat shield is considered only when the separation is yet to happen, once that the heat shield is isolated from the lander, the mass of the heat shield is considered to be zero.

Special Features:

- **Dynamic COM Calculation:** Uses updated fuel, dry, and heat shield masses to shift the COM vector and ensure realistic torque behavior.
- **Fuel Mass Integration:** Integrator block used to compute updated fuel mass from:

$$\frac{dm}{dt} = \frac{\text{Throttle} \cdot T_{\max}}{\text{exhaust velocity}}$$

8.3 Lander Dynamics

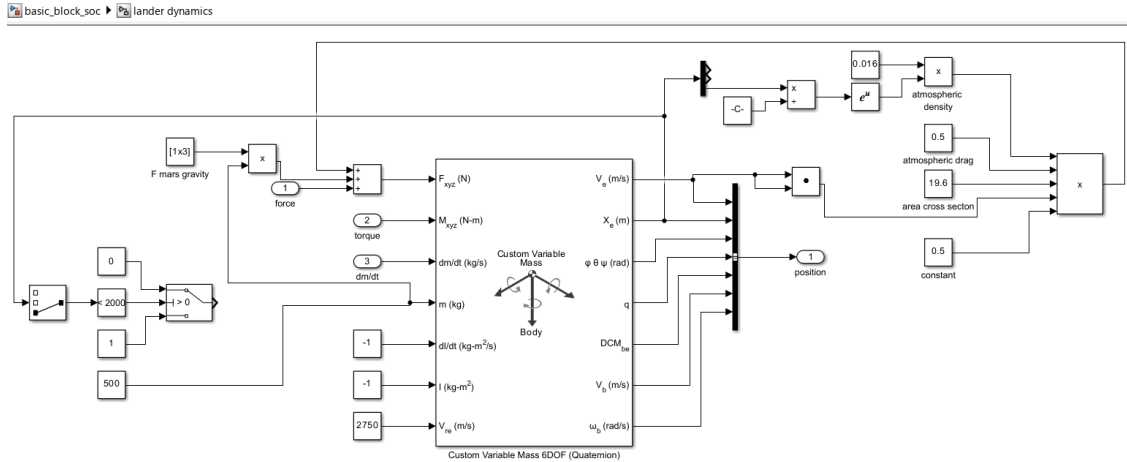


Figure 6: Lander dynamics subsystem

Objective: Simulate lander motion in Mars' gravitational field, accounting for thrust, mass depletion, and inertia.

Components and Functions:

- **Main block:** We used the custom variable 6DOF (Quaternion):
- **Inputs:**

- F_{xyz} : External force (gravity + thrust)
- M_{xyz} : External torque
- $\frac{dm}{dt}$: Mass depletion rate
- m : Total mass (fuel + dry)
- I : Inertia tensor
- $\frac{dI}{dt}$: Derivative of inertia tensor

• **Outputs:**

- Euler angles (ϕ, θ, ψ)
- Position (X_e)
- Velocity
- Angular velocity
- DCM (Direction Cosine Matrix)

Gravity Modeling: Mars gravity vector is scaled with current mass and added to the total force vector.

Special Features:

- **Atmospheric Drag Modelling:** Atmospheric Drag is modeled using the following formula:

$$D = C_d \times \frac{\rho V^2}{2} \times A$$

- ρ : The atmospheric density is exponentially modeled:

$$\rho(h) = \rho_0 e^{-h/H}$$

where $\rho_0 = 0.016$, $H \approx 10.6$ km (scale height for Mars), and h is altitude (negative of z position).

- A : Cross-sectional area
- C_d : Drag coefficient (assumed constant)

Velocity is squared and multiplied to get the drag force magnitude. Final drag force is in the opposite direction of velocity and added to the net force vector.

- **Dynamic Mass Handling:** Due to the separation of the heat shield, the mass of the lander changes. To compute the mass accordingly, we used a switch block. The custom variable 6DOF (Quaternion) block has a $\frac{dm}{dt}$ port, and the input from here is used to vary the mass dynamically as the fuel burns.
- **Quaternion 6DOF Block:** Avoids gimbal lock and gives accurate rotational behavior with orientation output in both quaternion and Euler formats.

- **Future extensions:** To simulate more realistic environmental conditions, we are planning to introduce wind disturbances in the lander dynamics model. Wind is modeled as an external velocity vector acting in the inertial (Mars-fixed) frame, and its effect is integrated by modifying the relative velocity between the lander and the atmosphere. In the atmospheric drag force that we calculated using the formula shown above, the velocity vector will be modified as $V_{rel} = V_{lander} - V_{wind}$. Here, V_{rel} = relative velocity vector

8.4 Sensors

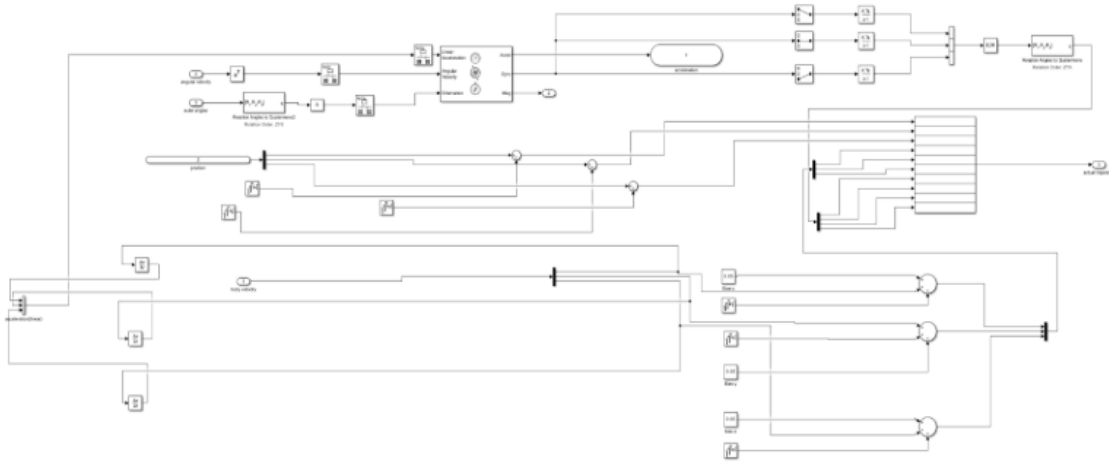


Figure 7: Sensor subsystem

8.4.1 Objective

The major objective of this subsystem is to simulate realistic onboard sensors that collect data from lander dynamics as inputs, they are fed to sensors. The intent is to get the most rational actual trajectory by adding noise.

8.4.2 Components and Major Blocks

Inputs Angular velocity, position, Euler angles, velocity are the inputs we are obtaining from the lander dynamics subsystem. All these inputs are in the form of a three element vector.

IMU Sensor (Inertial Measurement Unit) The IMU sensor available on Simulink has Accelerometer, Gyroscope and Magnetometer. It takes its inputs as Linear Acceleration as a $N \times 3$ matrix, on a separate note, we differentiate velocity to get linear acceleration and then input this to IMU, Orientation of the lander as a quaternion and Angular velocity as a $N \times 3$ matrix. The outputs from IMU are noisy linear acceleration, noisy angular velocity and noisy orientation as a quaternion.

Block Parameters: IMU

IMU

Simulate accelerometer, gyroscope, and magnetometer sensors.

The block outputs sensor measurements based on device motion. The inputs are in the local navigation frame. The outputs are in the local sensor body frame. The parameters on the Accelerometer, Gyroscope and Magnetometer tab can be set to match values on a sensor datasheet.

Parameters	Accelerometer	Gyroscope	Magnetometer
Maximum readings (m/s ²):	Inf		
Resolution ((m/s ²)/LSB):	0.001		
Constant offset bias (m/s ²):	[0.05, 0.04, -0.03]		
Axes skew (%):	[100, 0, 0; 0, 100, 0; 0, 0, 100] <3x3 double>		
Noise			
Velocity random walk ((m/s ²)/√Hz):	[0.005, 0.005, 0.005]		[0.005, 0.005, 0.005]
Bias instability (m/s ²):	[0.01, 0.01, 0.01]		[0.01, 0.01, 0.01]
Bias instability filter numerator coefficients:	fractalcoef().Numerator		1
Bias instability filter denominator coefficients:	fractalcoef().Denominator		[1, -0.5]
Acceleration random walk ((m/s ²)*√Hz):	[1e-4, 1e-4, 1e-4]		[0.0001, 0.0001, 0.0001]
Noise type:	double-sided		
Temperature Effects			
Bias from temperature ((m/s ²)/°C):	[0.01, 0.01, 0.01]		[0.01, 0.01, 0.01]
Temperature scale factor (%/°C):	[0.1, 0.1, 0.1]		[0.1, 0.1, 0.1]

OK
Cancel
Help
Apply

Figure 8: ss1

Block Parameters: IMU

IMU

Simulate accelerometer, gyroscope, and magnetometer sensors.

The block outputs sensor measurements based on device motion. The inputs are in the local navigation frame. The outputs are in the local sensor body frame. The parameters on the Accelerometer, Gyroscope and Magnetometer tab can be set to match values on a sensor datasheet.

Parameters	Accelerometer	Gyroscope	Magnetometer
Maximum readings (rad/s):	Inf		
Resolution ((rad/s)/LSB):	0.0001		
Constant offset bias (rad/s):	[0.01, 0.015, -0.02] [0.01,0.015,-0.02]		
Axes skew (%):	[100, 0.1, 0.2; 0.1, 100, 0.1; 0.2, 0.1, 100] <3x3 double>		
Bias from acceleration ((rad/s)/(m/s²)):	[0.02, 0.02, 0.02] [0.02,0.02,0.02]		
Noise			
Angle random walk ((rad/s)/√Hz):	[0.002, 0.002, 0.002] [0.002,0.002,0.002]		
Bias instability (rad/s):	[0.005, 0.005, 0.005] [0.005,0.005,0.005]		
Bias instability filter numerator coefficients:	fractalcoef().Numerator 1		
Bias instability filter denominator coefficients:	fractalcoef().Denominator [1, -0.5]		
Rate random walk ((rad/s)*√Hz):	[2e-5, 2e-5, 2e-5] [2e-05,2e-05,2e-05]		
Noise type:	double-sided		
Temperature Effects			
Bias from temperature ((rad/s)/°C):	[0.002, 0.002, 0.002] [0.002,0.002,0.002]		
Temperature scale factor (%/°C):	[0.1, 0.1, 0.1] [0.1,0.1,0.1]		

OK Cancel Help Apply

Figure 9: ss2

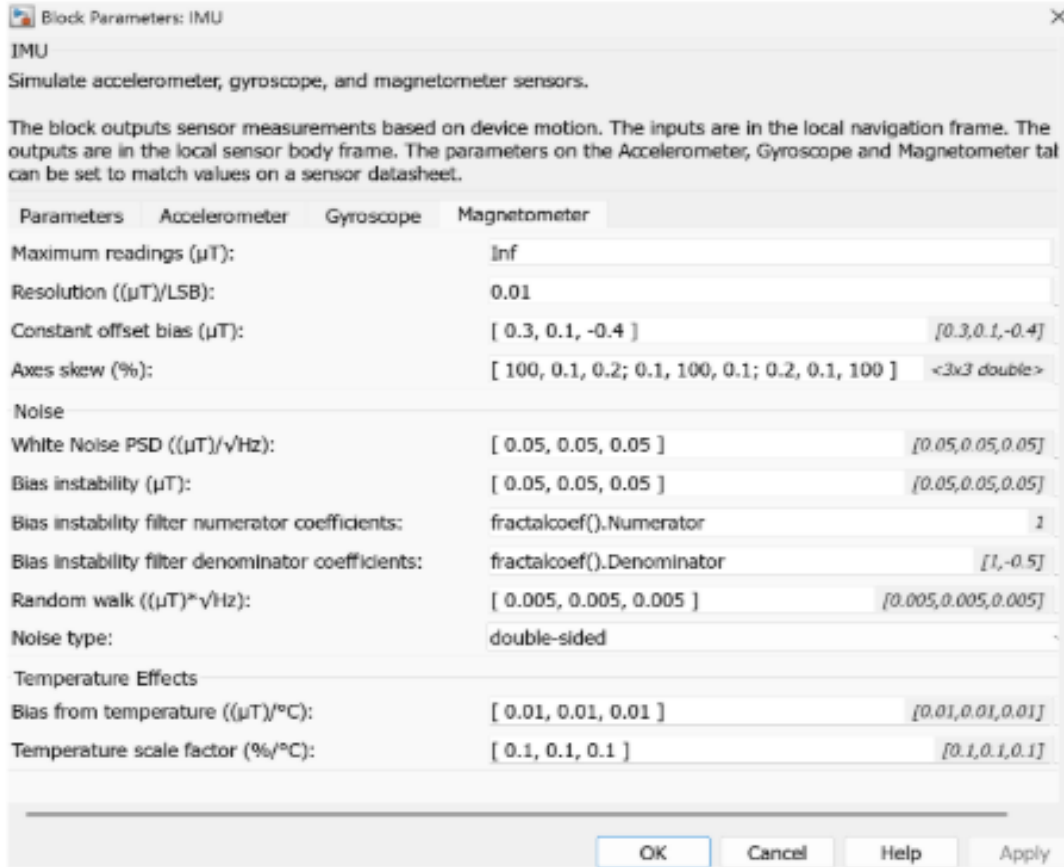


Figure 10: ss3

Noise Addition in IMU WHITE GAUSSIAN NOISE is added to both linear acceleration and angular velocity. This helps in simulating random fluctuations due to electronic noise. Similarly BIAS OFFSET is also added in the sensor which is used to simulate constant or time varying bias misalignment or sensor drift over time. Bias values can be set manually based on real sensor data sheets.

Here are the values added in the sensor: Constant Offset bias represents fixed bias (sensor miscalibration). These values are in the range of ± 50 , which is common for uncalibrated sensors. Axes skew is a 3×3 matrix which indicates slight axes misalignment. Velocity random walk simulates noise from sensor electronics. These values are in line with the datasheets for medium accuracy sensors (such as ADIS16470 or analog MEMS sensors). Also Bias instability reflects slow changing errors that occur over time. The value of 10 is typical for modeling bias drift in simulations, meanwhile, the Acceleration random walk is set to have very low values which helps in simulating extremely slow variations. This helps in forming a high stability system. We chose the noise to be double sided (noise is added symmetrically about 0). This is taken as a default assumption for Gaussian noise.

Rate Transition Block This block plays an important role in the sensor's subsystem, IMU block requires fixed step discrete inputs at a specified sample time and it does not support variable step or continuous signals directly. Whereas Simulink's 6DOF block runs

at continuous or variable sample rate, while IMU block inertially simulates sensor behaviour at discrete, fixed intervals. If we directly connect continuous signals to IMU, Simulink throws an error about mismatched sample times. Hence we use Rate Transition blocks which ensure proper data handoff between the continuous and the discrete time domains. Using this also helped us in eliminating solver related errors.

Orientation from Angular Velocity The IMU sensor gives linear acceleration from the accel output port, angular velocity from the gyro output port and earth's magnetic field measurements from the mag output port. The orientation in the form of quaternion does not directly come out as an output from the IMU sensor block. We will use simple scientific logic that is to integrate angular velocity to get the angles. In our simulation we used DISCRETE TIME INTEGRATOR to compute the angular displacement (Euler angles). We chose a discrete time integrator instead of standard (continuous) integrator block because the IMU outputs data at fixed discrete time intervals, a normal integrator block expects continuous signals and using it with discrete inputs can lead to errors in the simulation. This ensures that the integration aligns with the IMU's discrete sample time. Further we use a block called ROTATION ANGLES TO QUATERNION that helps in converting the Euler angles to quaternion.

Adding Noise to Position and Velocity In real world systems, position and velocity are never known with perfect accuracy. They should be estimated using noisy sensor measurements. To simulate realistic sensor behaviour we add a block called BAND-LIMITED WHITE NOISE. The position is added with band-limited white noise to add realistic Gaussian noise to the position measurements. The noise power is set to 0.1. The sample time is set to be 0.01 to align with other sensors. A fixed seed was used to maintain consistency across simulation runs. Apart from this the velocity is added with a constant bias of 0.05 this represents systematic offset that commonly arises in practical IMU or Doppler velocity sensors due to calibration errors. Unlike random noise, a bias is a persistent error that affects the measurement in a consistent direction.

Outputs The outputs of position, velocity and orientation are fused into a vector using a vector concatenate and then are passed into the control system.

8.5 Guidance

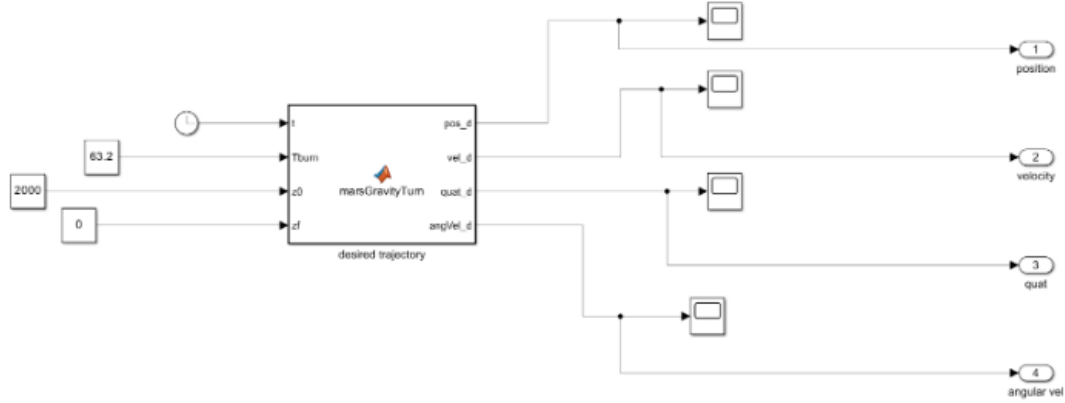


Figure 11: guidance subsystem

8.5.1 Objective

The Guidance subsystem provides the desired trajectory in terms of position, velocity, orientation and angular velocity by using a Matlab function block. This helps in governing the descent profile using a trajectory generation technique known as cubic polynomial interpolation. It will help us in defining the desired orientation based on the direction of descent and it will further calculate angular velocity and obtains a smooth gravity turn maneuver.

8.5.2 Major Components or Blocks

Matlab Function Block (marsGravityTurn) This block allows us to write various functions on Matlab and use them in our models. The term GravityTurn refers to the common strategy often used in landers. It should actually involve gravity so that the trajectory of vehicle is influenced by the gravity of the planet, but, in our model, the function does not include the term Mars gravity explicitly in the process of trajectory generation. The major aim here is to generate a desired trajectory for the Mars lander, assuming that the lander will follow the path in spite of the gravity. The function solely defines a kinematic perspective involving position, velocity and orientation. The gravity of Mars is actually considered in our Lander dynamics subsystem (6DOF Block). The function that we wrote to get the desired trajectory is:

```

function [pos_d, vel_d, quat_d, angVel_d] = marsGravityTurn(t, Tburn, z0, zf)
% Gravity-turn descent with simple constant deceleration
% Inputs: t = time (s), Tburn = total burn time
%       z0 = initial altitude, zf = final altitude (landing)
% Outputs: desired position, velocity, quaternion (orientation), angular velocity

% Mars gravity

% Altitude interpolation
if t < 0, t = 0; elseif t > Tburn, t = Tburn; end
tau = t / Tburn;

% Cubic altitude profile
z = z0 + (zf - z0) * (3 * tau^2 - 2 * tau^3);
dzdt = (zf - z0) * (6 * tau * (1 - tau)) / Tburn;
vel_d = [0; 0; dzdt];

% Position: vertical only in this model
pos_d = [0; 0; z];

% Orientation: align body +Z axis opposite velocity
if norm(vel_d) > 1e-3
    yaw = 0;
    pitch = atan2(-vel_d(3), 0.1+norm(vel_d)); % tilt roughly from descent
    roll = 0;
else
    yaw = 0; pitch = 0; roll = 0;
end
% Convert yaw/pitch/roll (ZYX) to quaternion
cy = cos(yaw/2); sy = sin(yaw/2);
cp = cos(pitch/2); sp = sin(pitch/2);
cr = cos(roll/2); sr = sin(roll/2);
q0 = cr*cp*cy + sr*sp*sy;
q1 = sr*cp*cy - cr*sp*sy;
q2 = cr*sp*cy + sr*cp*sy;
q3 = cr*cp*sy - sr*sp*cy;
quat_d = [q0; q1; q2; q3];

% Angular velocity: d(pitch)/dt about body Y axis
dpitchdt = ((- (zf - z0)*(6*(1 - 2*tau))/Tburn)) / ...
    sqrt(dzdt^2 + 1e-3);
angVel_d = [0; dpitchdt; 0];

end

```

Figure 12

Inputs of the Block

- t : Current Simulation time
- T_{burn} : Total burn time
- z_0 : Initial descent altitude
- z_f : Final landing altitude

In marsGravityTurn function, the time t is normalized, that is, as t goes from 0 to T_{burn} the normalized time τ goes from 0 to 1, using the below formula:

$$\tau = \frac{t}{T_{\text{burn}}}$$

This helps us in converting actual time t (which actually goes from 0 to T_{burn}) into a dimensionless quantity τ that will always stay in 0 to 1. This is done to make the use of polynomial interpolation (interpolation is a way to estimate the values between any known data points or in other words it is a way to find a function that fits the data points) easier.

Altitude Control We are going to generate the trajectory for our Mars lander using cubic polynomial interpolation. The function $z(\tau) = a\tau^3 + b\tau^2 + c\tau + d$ is a kind of smooth-step polynomial which is a type of Cubic Hermite Interpolation. Cubic interpolation polynomials help us in getting a smooth trajectory that starts and ends with zero velocity which is ideal for soft landing. Cubic Hermite interpolation is used to interpolate between two points with specified derivatives (here the derivatives are the velocities which are known).

Now we have the data:

$$\begin{aligned} z(0) = z_0 &\implies d = z_0 \\ z(1) = z_f &\implies a + b + c = z_f - z_0 \\ \frac{dz}{d\tau}(\tau = 0) = v_0 T_{\text{burn}} &\implies c = v_0 T_{\text{burn}} \quad (\text{here } v_0 \text{ is initial velocity}) \\ \frac{dz}{d\tau}(\tau = 1) = v_f T_{\text{burn}} &\implies 3a + 2b + c = v_f T_{\text{burn}} \quad (\text{here } v_f \text{ is final velocity}) \end{aligned}$$

Now from the above conditions we solve to get the values of a , b , c and d . They are as follows:

$$\begin{aligned} a &= (v_f + v_0)T_{\text{burn}} - 2(z_f - z_0) \\ b &= z_f - z_0 - v_0 T_{\text{burn}} - a \\ c &= v_0 T_{\text{burn}} \\ d &= z_0 \end{aligned}$$

Also, we have the condition that initial velocity and the final velocity are 0, that is:

$$v_0 = 0, \quad v_f = 0$$

Now, substituting all the values in the equation we get:

$$z = z_0 + (z_f - z_0)(3\tau^2 - 2\tau^3)$$

Velocity Control Meanwhile the velocity (in the code represented as dz/dt but actually $\frac{dz}{d\tau}$) is calculated by simple differentiation of the above equation:

$$\frac{dz}{dt} = \frac{(z_f - z_0)(6\tau(1 - \tau))}{T_{\text{burn}}}$$

Orientation Control In our implementation the orientation is defined using pitch because:

- The descent path is primarily in the vertical direction (z-axis), with some horizontal (x) drift. That means:
- Yaw (rotation around z-axis) is irrelevant, as there's no change in horizontal direction.
- Roll (rotation around x-axis) doesn't contribute to pointing the thrust vector correctly.
- Only pitch (rotation about y-axis) determines how the main engine is angled to oppose motion.

We define pitch as:

$$\text{pitch} = 2(-v_d(3), 0.1 + \|v_d\|)$$

Here,

- $v_d(3)$ is the z-component of velocity (i.e., vertical speed).
- $-v_d(3)$ means we want to tilt against the velocity vector (opposite direction).
- $\|v_d\|$ is the total speed, used to normalize the pitch angle.

In the denominator 0.1 is added to avoid the singularities or the extreme angles near the zero velocity points. This returns a small pitch angle when the descent is steep, mostly vertical. It has a larger pitch when the lander glides forward with more horizontal motion. It estimates the angle between the body's z-axis and the opposite of the velocity vector.

Converting θ , ϕ , ψ (pitch, roll, yaw respectively) into Quaternion We have a set of equations that are used to convert Euler angles into quaternion they are: Hence we get the output as a quaternion.

Outputs We get the outputs as position (3D), velocity (3D) and orientation (quaternion).

9 Challenges in PID Tuning

The initial attempt to tune the PID controller using the PID Tuner showed an error: "The plant cannot be linearized." This was most likely because the plant dynamics were highly nonlinear, mainly due to the Euler equations that describe its rotational motion. Since the PID tuner needs a linear model to work with, it was unable to perform the tuning directly.

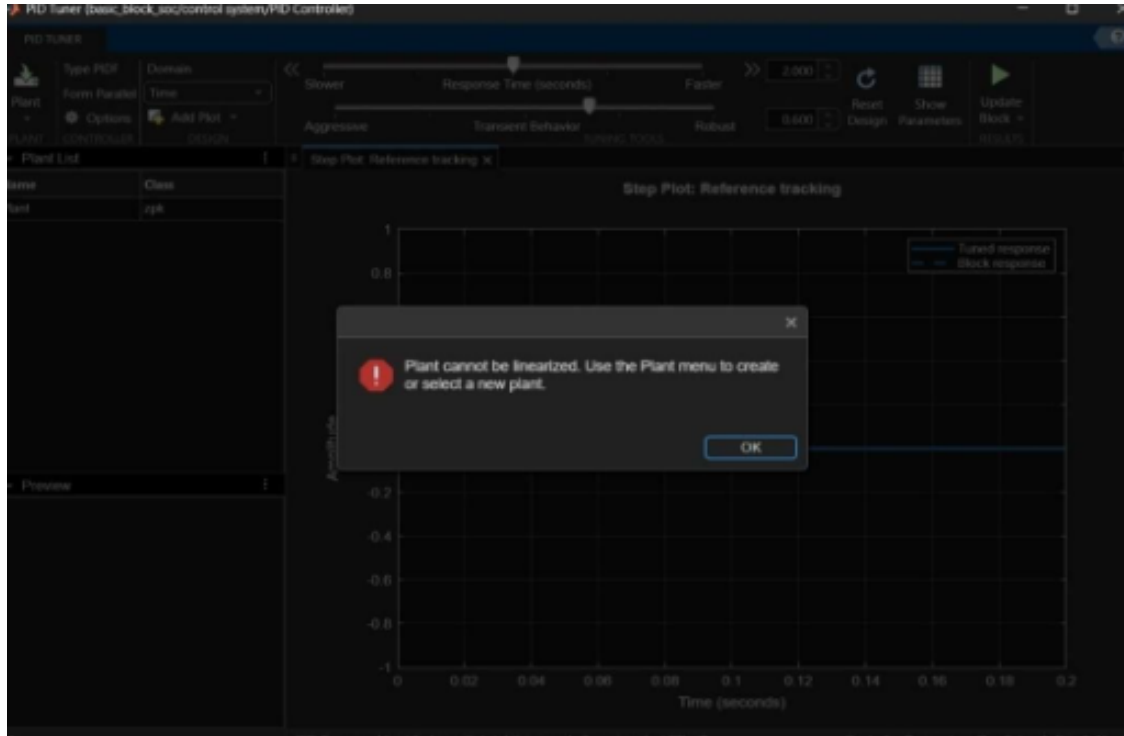


Figure 13: non linear plant

9.1 Steps Taken to Fix the Issue

1. Identify a new plant model – A new mathematical model was created to capture the nonlinear behavior of the system better.
2. Collect input-output data – Different input signals were applied to the system, and the corresponding outputs were recorded to build a useful dataset.
3. Fit the mathematical model – The collected data was used to fit a model that accurately describes the plant's dynamics.
4. Tune the PID using the new model – The newly identified plant model was then used to calculate the PID gains, leading to improved tuning and better overall controller performance.

Despite implementing this approach, the results obtained using the PID controller were not satisfactory. This could be attributed to the requirement of achieving an exact Euler orientation of $[0, 0, 0]$. To address this, the condition was relaxed by defining an allowable error range within which the PID controller remains inactive. This modification helps to avoid unnecessary oscillations and overshoot that arise from attempting to achieve perfect orientation.

Even though there was a slight error in orientation, the system was still able to decelerate from 77 to 4. Moreover, the thrust in the z-direction remained largely unaffected.

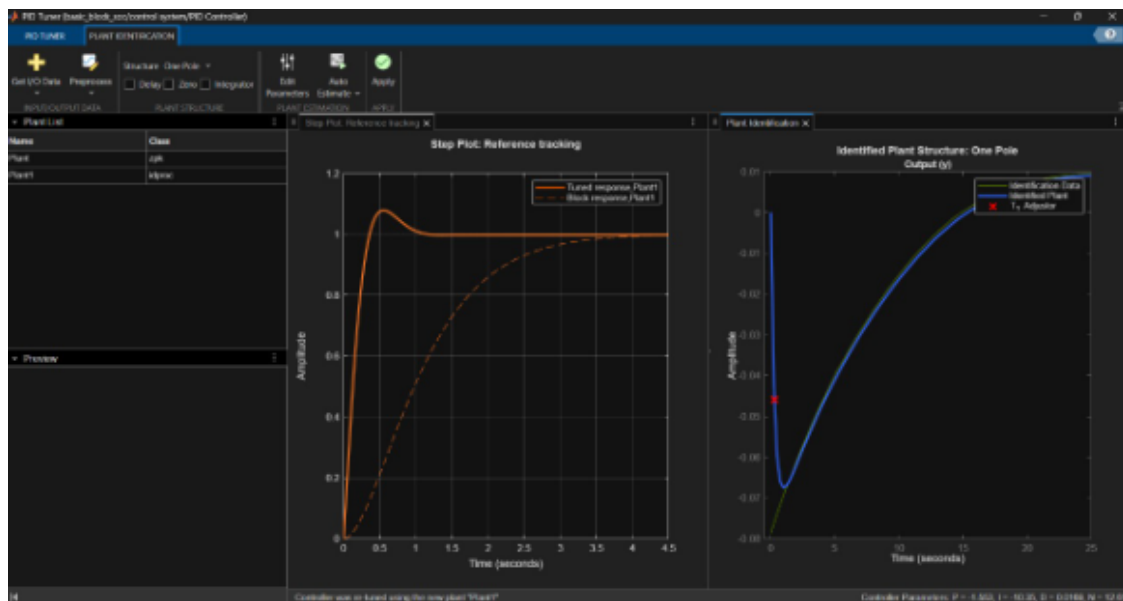


Figure 14: troubleshoot method

10 Initial Condition or Block Parameters

Block Parameters: Custom Variable Mass 6DOF (Quaternion) X

6DOF EOM

Integrate the six-degrees-of-freedom equations of motion in body axes.

Main State Attributes

Units: Metric (MKS) v

Mass type: Custom Variable v

Initial position in inertial axes $[X_e, Y_e, Z_e]$:

[0 0 -2000] ...

Initial velocity in body axes $[U, v, w]$:

[0 0 77] ...

Initial Euler orientation [roll, pitch, yaw]:

[0.05 0.1 0] ...

Initial body rotation rates $[p, q, r]$:

[0 0 0] ...

☒ Include mass flow relative velocity

OK Cancel Help Apply

Figure 15: initial conditions

The dynamics of the vehicle were configured with particular initial conditions at the beginning of the descent. Given that the z-axis is positive in the downward direction, the vehicle started at an inertial position of $[0, 0, -2000]$ meters, or 2000 meters above the Martian surface. With an initial velocity of $[0, 0, 77]$ m/s in the body frame, it was travelling at a straight downward speed of 77. With small roll, pitch, and yaw angles of $[0.05, 0.1, 0]$ radians, the initial orientation was slightly distorted.

11 Results

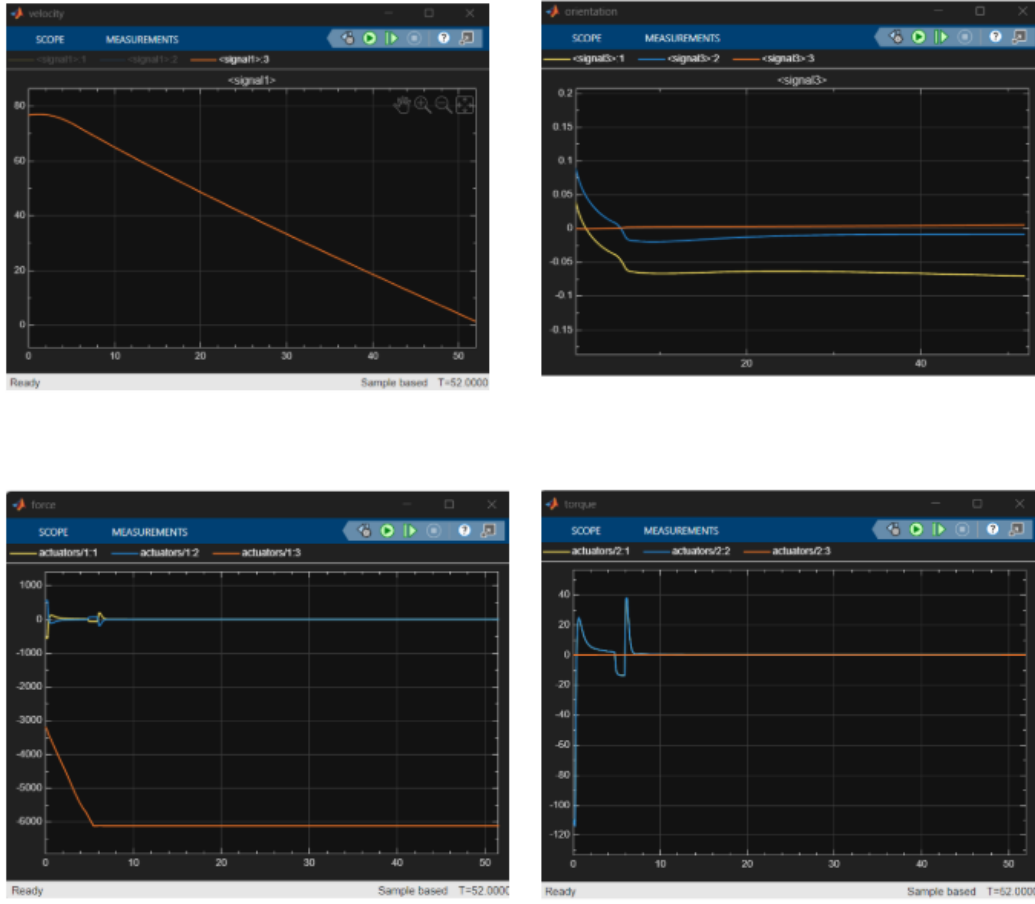


Figure 16: graphs subsystem

11.1 Velocity

Throughout the descent phase, the velocity plot shows a steady and even decrease in the lander's downward speed. The velocity starts out at about 77 and gradually drops without any abrupt swings, eventually falling below 10 by the end of the 52-second simulation. The lander will safely approach the surface due to this steady profile, indicating a well-managed and controlled descent trajectory.

11.2 Orientation

The orientation plot shows how the attitude of the lander changes as it descends. First, there are visible roll and pitch tilts, as well as a small yaw deviation. These angles gradually approach zero over time, indicating that the lander is aligning itself with the intended orientation. There are still some slight residual offsets, but they are negligible and not harmful to stability, suggesting that the attitude control system operates efficiently.

11.3 Force and Torque

The force plot captures the engines' response throughout the descent. At the beginning, the x and y thrust components rise noticeably, producing torque to correct the lander's initial tilt and align it with the desired orientation. During this phase, the engines work together to stabilize attitude while slowing the downward motion. As the orientation settles, the x and y components fade, approaching zero. In the final stage, only the z-direction thrust remains, holding a steady negative value to ensure a smooth deceleration.

The torque plot mirrors this pattern. Early in the descent, spikes appear in the torque values, marking strong corrective actions against the initial tilt. These oscillations gradually shrink as stabilization completes. By the end, the torques hover near zero, showing that only minimal adjustments are needed once the lander is properly aligned.

12 Designing the Lander on SolidWorks

This is the CAD design of the gimbal rings and the engine, designed to provide descent thrust.

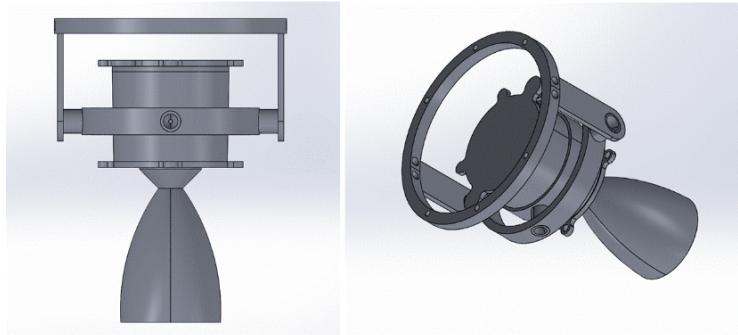


Figure 17

The above part is the combustion chamber where the fuel and the oxidizer burned and converted into the high pressure gas. This pressure acts as thrust which helps the lander to control motion. The exit nozzle diameter is about 0.49 cm and the minimum throat diameter is about 0.078 m. The combustion chamber is of height 0.3 m and diameter of 0.4 m and thickness of 0.02 m. The inner shape of the chamber is a cylindrical mounted cone which has an opening at the end. The reasonable expansion ratio of Mars lander is 40 (taken from real world examples).

Let the required thrust be 6220 N,

The combustion chamber pressure is 0.8 MPa (taken from real world examples),

Thrust coefficient (C) is 1.6,

Specific heat ratio for MMH+NTO = 1.2,

Mars atmospheric pressure is 610 Pa.

12.1 Throat Area and Diameter Calculations

$$\text{Area of throat} = \frac{F}{P \cdot C} = \frac{6220}{800000 \cdot 1.6} = 0.00485 \text{ m}^2$$

$$\text{Area} = \frac{\pi d^2}{4} \Rightarrow d = \sqrt{\frac{4 \cdot 0.00485}{\pi}} = 0.0786 \text{ m}$$

12.2 Exit Nozzle Diameter

$$\text{Area of exit nozzle} = \text{Expansion Ratio} \cdot \text{Area of throat} = 40 \cdot 0.00485 = 0.194 \text{ m}^2$$

$$\text{Area} = \frac{\pi d^2}{4} \Rightarrow d = \sqrt{\frac{4 \cdot 0.194}{\pi}} = 0.496 \text{ m} = 49.6 \text{ cm}$$

12.3 Isentropic Flow Relation for Nozzle Exit Pressure

We use the isentropic flow relation to determine the pressure at the nozzle exit:

$$\frac{p}{p_t} = \left(1 + \frac{\gamma - 1}{2} M^2\right)^{-\frac{\gamma}{\gamma - 1}}$$

Where:

- p = exit pressure
- p_t = chamber (total) pressure = 800 kPa
- $\gamma = 1.2$ (for air approximation)
- $M = 5.31$ (Mach number at nozzle exit)

From the equation and substitution:

$$\frac{p}{p_t} = 0.000812 \quad \Rightarrow \quad p = 0.000812 \times 800000 = 649.6 \text{ Pa} \approx 0.65 \text{ kPa}$$

This pressure drop is expected in supersonic expansion and supports nozzle efficiency in the thin Martian atmosphere.

12.4 Gimbal Mechanism

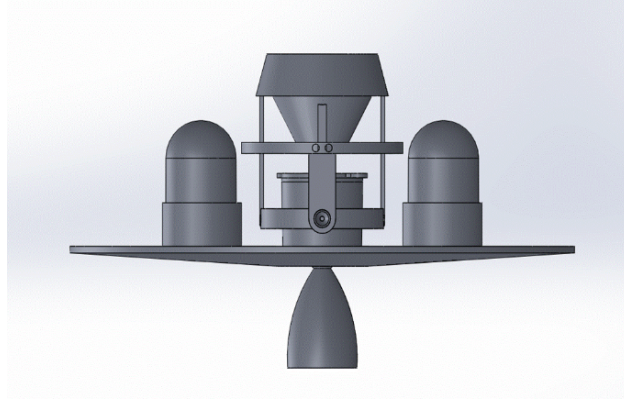


Figure 18

The gimbal mechanism rotates in pitch and yaw directions. The inner ring rotates in yaw and the outer ring rotates in pitch direction.

The upper ring, which is attached to the lander, is of diameter 0.8 m. The particular reason for this diameter is to allow free gimbal rotation. A convergent-divergent nozzle is used due to its high efficiency in thin atmospheres like Mars. It narrows at the throat and widens at the exit, increasing the exhaust velocity to supersonic.

12.5 Propellant Tank Placement and Volume

The propellant tanks are placed on either side of the engine to place the centre of gravity low. The volume of propellant tank is about 148 L.

12.5.1 Calculating Fuel Tank Volume

Let $v = 400$ m/s (assumption)

Mars gravity $g = 3.721$ m/s²

Descent altitude = 2000 m

$I_{sp} = 310$ s

Using the Tsiolkovsky Rocket Equation:

$$\Delta v = I_{sp} \cdot g \cdot \ln \left(\frac{M_0}{M_f} \right)$$

- $M_0 = 1194$ kg
- Solving for M_f : $M_f = 852$ kg
- Propellant mass = $M_0 - M_f = 342$ kg

Mixture Ratio: Given $O/F = 1.65$:

$$f + 1.65f = 342 \Rightarrow f = 129 \text{ kg}, \quad o = 212 \text{ kg}$$

Volume of Tanks:

- MMH density $880 \text{ kg/m}^3 \rightarrow V_f = \frac{21.27}{880} = 146.6 \text{ L}$
- NTO density $1440 \text{ kg/m}^3 \rightarrow V_o = \frac{35.09}{1440} = 147.2 \text{ L}$

The tank volume is extended to 148 L for safety margins.

Tank and Engine Placement Considerations

Tanks are cylindrical with hemispherical ends. MMH (fuel) and NTO (oxidizer) are used as hypergolic propellants—igniting on contact—which are common for Mars missions due to their storability and reliability. Tanks are placed close to the engine to avoid torque and are shielded from heat and vibration. This avoids thermal radiation risk and pressure imbalances.

Volume is based on the specific impulse of the propellant (310 s). Side-mounted tanks help maintain balance, reduce thermal stress, and simplify structural design.

Lander Body Design

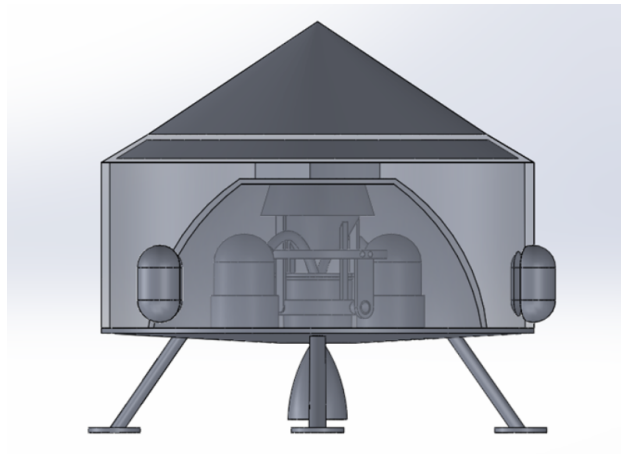


Figure 19

The cone on the lander is used for the parachute housing (disk-gap-band parachute, approx. 35 L volume). Cylinders placed symmetrically hold sensors for temperature and radiation data.

The IMU is aligned with the central axis and located above the engine for protection. We use three landing legs for stability and simplicity. A tripod provides stable support with less weight and complexity. Three footpads distribute landing loads.

Landing legs are positioned such that engine rotation is unhindered. An electromechanical actuator mounted above the engine controls gimbal movement. Shielding around tanks and engine prevents overheating and vibrations from the Martian atmosphere.

References

13 Control Authority

Control authority refers to the Mars lander's ability to adjust its orientation and position during various phases of its descent and landing. The control over the powered descent during landing is the critical phase where the lander should control attitude and descent rate using the main gimbal engine. This engine produces pitch and yaw by directing thrust away from the vehicle's centreline. This enables precise control during soft landing. The gimbal and the engine are aligned with the central axis. If the lander slightly changes its direction then the gimbal engine rotates around and aligns through our desired path. The sensors placed at the different positions measure pressure, attitude, temperature and weather conditions of the Martian atmosphere this helps the lander to descend safely according to the conditions.

13.1 Control Authority Analysis

- Lander mass $m = 1194$
- Lander has a central main engine with thrust $T = 6220$
- Center of mass is at $(0, 0, 0.4)$ m
- Gimbal angle max = ± 10
- Distance from gimbal pivot to center of mass (lever arm) = 0.4

Torque is calculated as:

$$\text{Torque} = r \cdot T \cdot \sin(\theta)$$

where $r = 0.4$ (distance from engine to center of mass), $T = 6220$ (engine thrust), $\theta = 10$ (gimbal angle):

$$\text{Torque} = 0.4 \cdot 6220 \cdot \sin(10^\circ) = 432$$

The gimbal engine can apply about 432 of torque to the lander. Required angular acceleration is:

$$\alpha = \frac{\text{Torque}}{I}$$

To determine moment of inertia similar to a cylinder:

$$I = \frac{m(3R^2 + h^2)}{12}$$

where $R = 1.25$ (radius), $h = 1.8$ (height of the lander):

$$I = 789$$

$$\alpha = \frac{432}{789} = 0.55$$

Angular acceleration = 0.55.

Time to Rotate a Given Angle To rotate by θ radians from rest under constant angular acceleration:

$$\theta = \frac{\alpha t^2}{2}$$

To rotate by 10 = 0.1745:

$$t^2 = \frac{2 \cdot 0.1745}{0.55}, \quad t = 0.79$$

Margin of Control Authority Max torque available: 432. Required torque to rotate within 1 second:

$$T = I \cdot \alpha = 789 \cdot \frac{2 \cdot 0.1745}{1^2} = 275$$

Hence the torque available is greater than the required torque. We can get control over the lander.

- Rotation matrices: <https://ntrs.nasa.gov/api/citations/20240012525/downloads/20240012525-rev01.pdf>
- Atmospheric drag: <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/drag-equation/>
- Atmospheric density: https://en.wikipedia.org/wiki/Density_of_air
- Isentropic flow: <https://www.grc.nasa.gov/www/BGH/isentrop.html>
- Tsiolkovsky Equation: <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/ideal-rocket-equation/>

Though our model is working without the guidance and sensor subsystems, the one that includes guidance and sensor subsystems is giving us problems. These are because we were unable to tune the PID properly as it is a nonlinear system.